

Telco

Summary of the Results

Summary

In this task, I developed several supervised learning models to predict whether a telecom customer will churn. The models were trained on structured tabular data, and multiple machine learning approaches were compared, including Logistic Regression, Random Forest, Gradient Boosting, and a neural network (MLP).

Among all models, tree-based models such as Gradient Boosting(AUC=0.8434) achieved the best performance, with the highest AUC score. Overall, the results show that traditional machine learning models are highly effective for tabular data, often outperforming neural networks in this setting.

Results and Evaluation

The models were evaluated using:

1. Area Under the ROC Curve (AUC)
2. Accuracy

The comparison results show that:

- Gradient Boosting achieved the highest AUC
- Random Forest also performed strongly
- Logistic Regression provided a strong baseline
- The neural network achieved reasonable performance but did not outperform tree-based models

These results indicate that tree-based models are well-suited for structured tabular datasets.

Approach

Data Preparation

The Telco Customer Churn dataset was used.

The target variable Churn was converted into binary format (1 = churn, 0 = not churn).

The customerID column was removed since it does not contribute to prediction.

The dataset contains both numerical and categorical features, so preprocessing was required.

Preprocessing Pipeline

Preprocessing pipeline to handle mixed data types:

1. Numerical features:
 - Missing values were filled using the median

- Features were scaled using StandardScaler
- 2. Categorical features:
 - Missing values were filled using the most frequent value
 - One-hot encoding was applied to convert categories into numerical form

This pipeline ensures that all features are properly formatted for machine learning models.

Model Architecture and Selection

Multiple models were trained and compared:

- Logistic Regression
- Random Forest
- Gradient Boosting
- SGDClassifier
- Keras Neural Network (MLP)

The neural network consists of:

- Multiple Dense layers with ReLU activation
- Dropout layers to reduce overfitting
- A final sigmoid output layer for binary classification

The model was trained using the Adam optimizer and binary cross-entropy loss.

Training Strategy

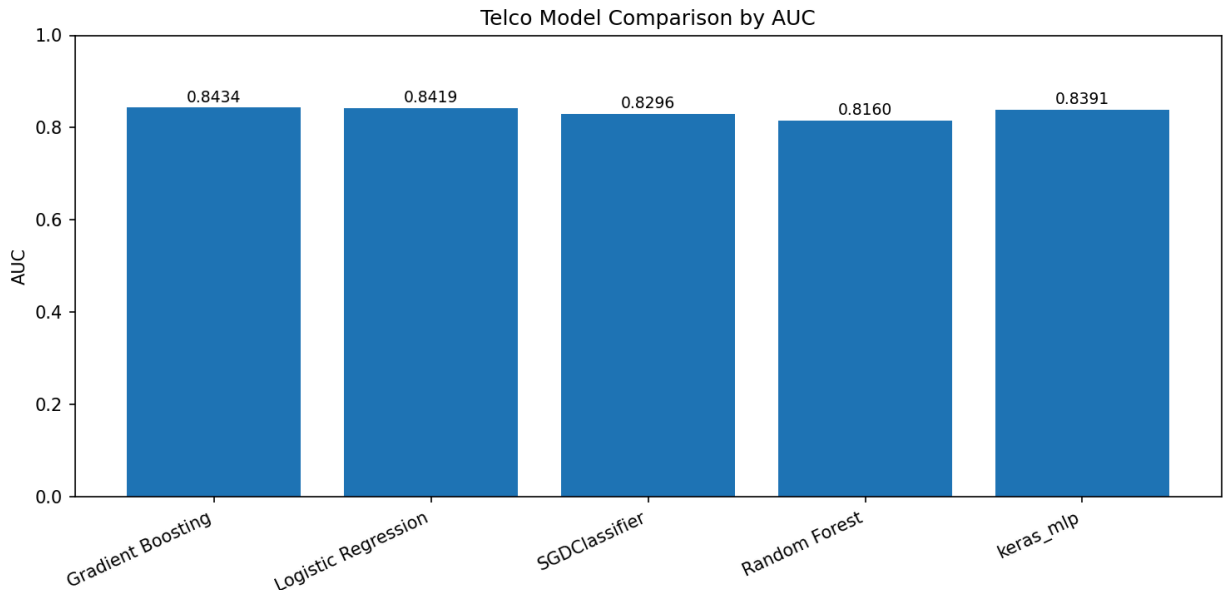
- The dataset was split into training and validation sets using an 80/20 split.
- Stratified sampling was used to preserve class distribution.
- Model performance was evaluated using validation AUC and accuracy.

Additionally, gradient tracking was implemented for the neural network to monitor training stability.

Detailed Findings, with Screenshots and Explanations

Model Performance

- Tree-based models (especially Gradient Boosting) achieved the highest AUC scores.
- Logistic Regression performed well as a baseline model.
- The neural network achieved competitive performance but did not surpass tree-based models.



The figure shows the AUC comparison across all models. Gradient Boosting achieved the highest performance (AUC = 0.8434), slightly outperforming Logistic Regression and the neural network model. While the Keras MLP model performed competitively (AUC = 0.8391), it did not surpass the best tree-based model. Overall, the results indicate that traditional machine learning models, particularly Gradient Boosting, are highly effective for structured tabular data.

Gradient Analysis (Neural Network)

- The gradients remained stable throughout training.
- No evidence of vanishing or exploding gradients was observed.

Key Observations

- Tree-based models perform better on tabular data because they can naturally handle feature interactions.
- Neural networks require more tuning and larger datasets to outperform traditional models.
- Logistic Regression provides strong interpretability but slightly lower performance.

Outlook / Future Enhancements, Risks

Future Enhancements

- Perform hyperparameter tuning for tree-based models
- Use feature selection or feature engineering

- Try more advanced models such as XGBoost or LightGBM
- Improve neural network architecture with more layers or regularization

Risks and Limitations

- The dataset may not capture all real-world customer behavior factors
- Class imbalance may affect model performance
- Neural networks may underperform due to limited dataset size

References, incl. AI usage statement

Tool used: ChatGPT

How I used AI:

Used AI to help understand concepts, debug code, and improve clarity and organization of the report.

How I verified outputs:

All results were tested by running the code and validated using metrics (AUC, RMSE, MAE) and visualizations.

Approximate percentage of code written by AI: ~20%

Bitcoin

Summary of the Results

Summary

This project built an LSTM forecasting model to predict Bitcoin daily high prices using a 30-day lookback window. Trained on 2019–2022 data and tested on 2023–2025 data, the model achieved low RMSE and MAE, showing that it captures the overall price trend effectively. While the model performs well in capturing overall trends, it may not be suitable for precise short-term trading decisions due to the high volatility of Bitcoin prices.

Results and Evaluation

The model was evaluated on the test dataset using:

1. Root Mean Squared Error (RMSE)
2. Mean Absolute Error (MAE)

The model achieved a test RMSE of 1207 and MAE of 826. These results indicate that the model can track the overall trend of Bitcoin prices, although short-term spikes remain difficult to predict.

```
Test RMSE: 1207.3339
Test MAE : 826.7277
```

Approach:

Data Preparation

1. Historical Bitcoin price data was used, and only the Date and High columns were retained, as instructed.
2. The dataset was sorted chronologically to preserve temporal order.
3. The data was split into:
 Training set: 2019–2022
 Test set: 2023–2025
4. MinMax scaling was applied to normalize the data into the range [0, 1], which helps improve training stability and model performance.
5. The time series data was transformed into a supervised learning format using a sliding window approach. A 30-day lookback window was used to predict the next day's high price.

Model Architecture

An LSTM-based neural network was implemented using Keras. The model consists of:

- An LSTM layer with 64 hidden units to capture temporal dependencies

- A Dropout layer (rate = 0.2) to prevent overfitting by randomly dropping 20% of neurons during training
- A Dense layer with ReLU activation for feature transformation
- A final Dense layer with one unit for regression output

The model was trained using the Adam optimizer with a learning rate of 0.001. The loss function used was Mean Squared Error (MSE), and Mean Absolute Error (MAE) was used as an additional evaluation metric. LSTM was chosen for this task because it is specifically designed to capture temporal dependencies in sequential data. Unlike traditional regression models, LSTM can learn patterns across time steps, making it more suitable for time-series forecasting.

Training Strategy

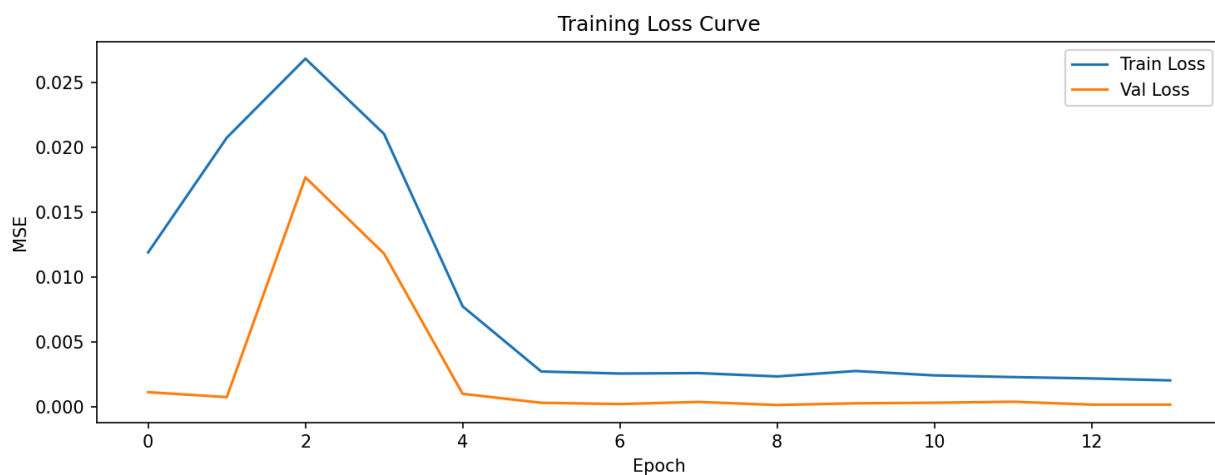
1. The data was not shuffled during training to preserve the time-series structure.
2. The last 10% of the training data was used as a validation set.
3. EarlyStopping was applied to prevent overfitting by stopping training when validation loss stopped improving.
4. ReduceLROnPlateau was used to dynamically adjust the learning rate when the validation performance plateaued.

Detailed Findings, with Screenshots and Explanations

Model Performance

The training loss decreased steadily over epochs, indicating successful learning.

The validation loss followed a similar trend, suggesting that overfitting was controlled.



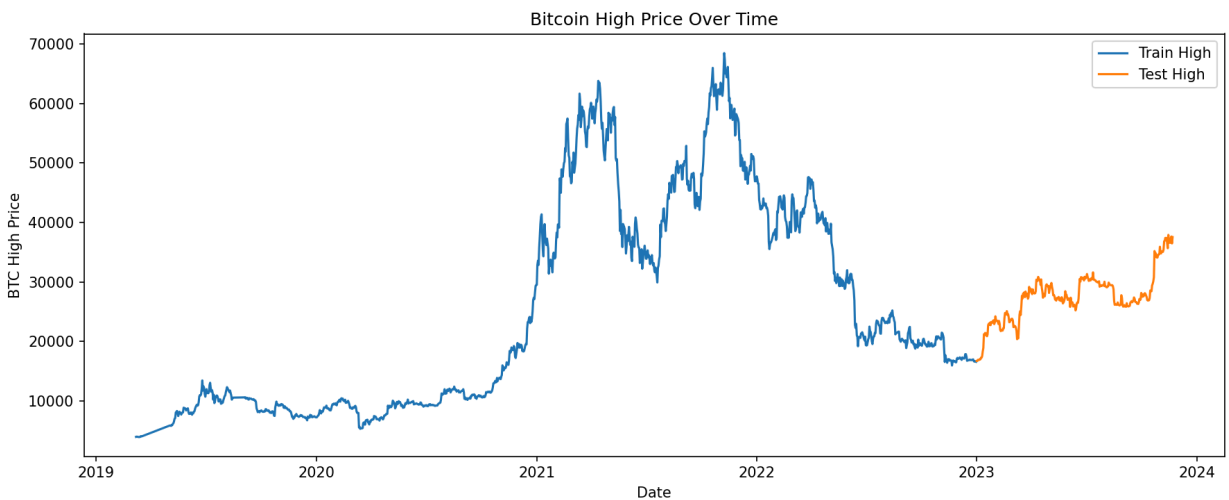
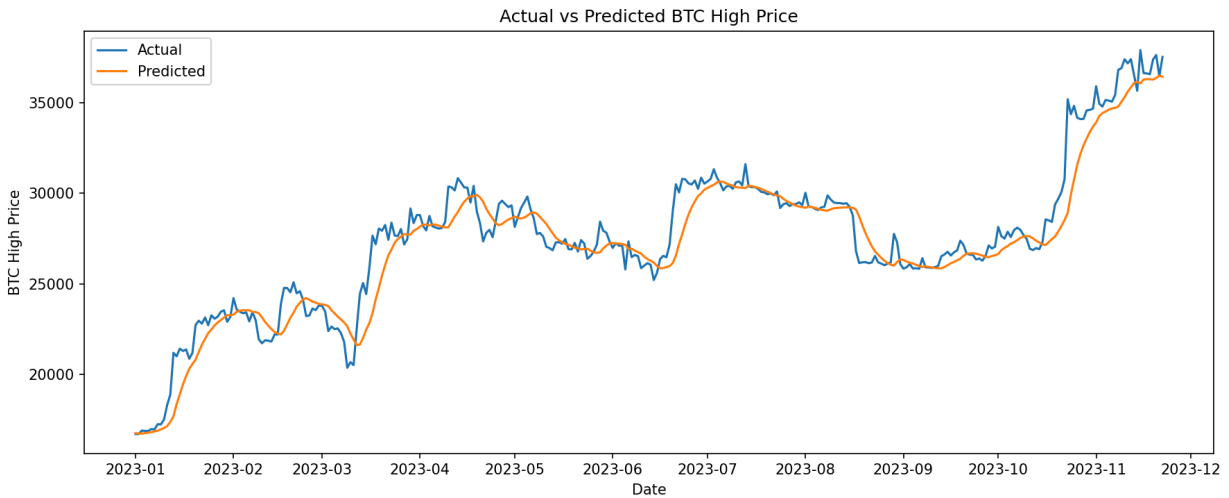
The training loss decreased steadily over epochs, indicating successful learning.

The validation loss followed a similar trend, suggesting that overfitting was controlled.

Prediction Results

The predicted Bitcoin prices closely follow the actual price trend.

The model captures long-term trends well, but some short-term fluctuations are not perfectly predicted.



Key Observations

The model performs well on trend prediction but struggles with sudden price spikes or drops.

The use of a 30-day lookback window provides sufficient historical context for capturing general patterns.

Scaling the data significantly improves training stability and model performance.

Outlook / Future Enhancements, Risks

Future Enhancements

- Testing different lookback window sizes (e.g., 7 days, 60 days)
- Adding additional features such as volume, open price, and close price
- Using deeper or stacked LSTM architectures
- Incorporating attention mechanisms to better capture important time steps

Risks and Limitations

- Bitcoin prices are highly volatile and influenced by external factors not included in the model.
- The model may not generalize well during extreme market conditions.
- Using only one feature (High price) limits the model's ability to capture full market dynamics.

References, incl. AI usage statement

References

Course materials from DSCI 352

Keras and TensorFlow documentation

Scikit-learn documentation

AI Usage Statement

Tools used: ChatGPT

How AI was used:

AI was used to assist with understanding LSTM concepts, debugging code, and improving the clarity and structure of the report.

How outputs were verified:

All code was reviewed, tested, and validated against expected outputs. Model results were checked using RMSE and MAE, and visualizations were used to confirm correctness.

Approximate percentage of code written by AI: <20%

Comparative Insight

This project includes two fundamentally different supervised learning tasks: Telco customer churn prediction and Bitcoin price forecasting. These tasks highlight key differences in data structure, model design, and evaluation methods.

The Telco task uses structured tabular data, where each row represents an independent customer. In contrast, the Bitcoin task is a time-series problem, where each data point depends on previous observations. As a result, different modeling approaches are required.

Traditional machine learning models such as Logistic Regression, Random Forest, and Gradient Boosting performed well on the Telco dataset, with Gradient Boosting achieving the highest AUC. These models are effective for tabular data because they can naturally handle feature interactions and do not require sequential information.

On the other hand, the Bitcoin forecasting task requires modeling temporal dependencies. An LSTM neural network was used because it can capture sequential patterns over time, which traditional models cannot directly handle.

Additionally, the evaluation metrics differ between the two tasks. The Telco model uses AUC and accuracy for classification, while the Bitcoin model uses RMSE and MAE for regression.

Overall, this comparison reflects model selection must be aligned with the data type and problem structure. Tree-based models are well-suited for tabular data, while deep learning models such as LSTM are more appropriate for time-series forecasting.